

继承

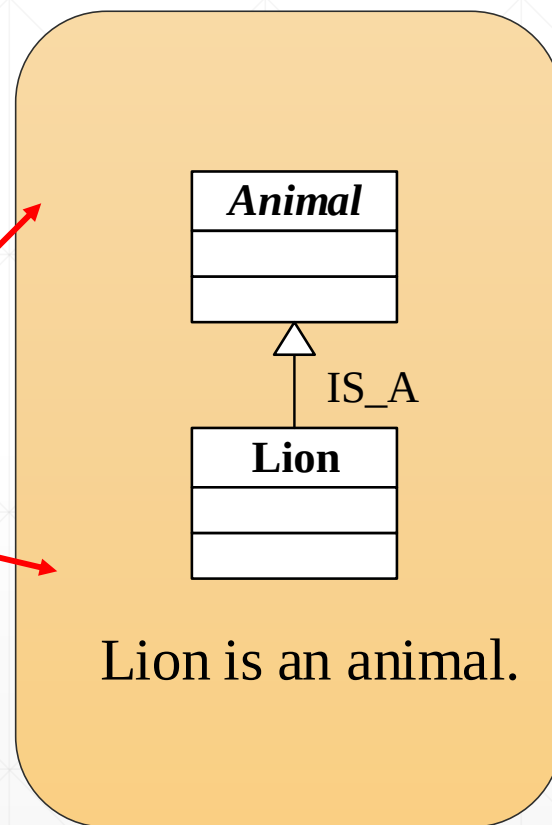
北京理工大学计算机学院
金旭亮



什么是继承？

继承是对现实生活中的“分类”概念的一种模拟。

狮子拥有动物的一切基本特性，但同时又拥有自己独特的特性，这就是“继承”关系的重要特性：通常简称为“**IS_A**”关系。



继承的语法

```
class 子类名 extends 父类名 {  
    .....  
}
```

```
Parent    --> Child  
Super Class --> Sub Class  
Base      --> Derived
```

父类 (parent class)
超类 (super class)



通常指直接上级

基类 (base class)



通常指包括直接上级在内的“上级的上级”

在Java中实现继承

```
class Animal
```

```
{  
}
```

```
class Lion extends Animal
```

```
{  
}
```



父类 (parent class) /
基类 (base class)



子类 (child class)



子类自动拥有父类声明为public和protected的成员，这就是继承特性的体现之一。

继承条件下类的访问权限



1. `public`: 外界可自由访问
2. `private`: 外界不可访问
3. `protected`: 同一包中的子类都可以访问, 另一包中的子类 (派生于同一个父类) 也可以访问
4. `default`: 如果不指明任何权限, 则默认同一包中的类可以访问

动手实验：继承条件下的构造方法调用



阅读并运行TestInherits.java示例，观察示例输出，注意总结父类与子类之间构造方法的调用关系。

在Parent构造方法中取消被注释掉的语句，让Parent构造方法通过super关键字显式用GrandParent的另一个构造函数。注意这句调用代码的位置。

通过以上试验，你得到了哪些结论呢？

结论



创建子类对象时，基类的构造方法会被自动地调用。



如果要使用`super`关键字调用基类的特定构造方法，则必须是子类构造方法中的第一条语句。

思索：

为什么子类的构造方法在运行之前，必须调用基类的构造方法？能不能反过来让基类调用子类的构造方法呢？

提示：

构造方法的主要作用是什么？从这个方面去想！



结论



类的构造方法，其主要职责是初始化类的字段。



依据继承的特性，子类代码是可以访问父类成员的，在某些情况下，子类字段的初始化可能需要访问基类的字段，因此，为保证子类字段在任何情况下都能顺利地被初始化，先初始化基类的字段，让其有确定的值，是合理的。

不允许继承的类

Java中final的功用

```
final class 类名 {  
}
```

1. 以final声明的变量不允许更改。
2. 以final声明的方法不允许覆盖。
3. 以final声明的类不能再派生出子类。

利用final，我们可以设计出一种特殊的“只读”的“不可变类（immutable class）”……



何为“不可变的类”？

创建“不可变的类”的对象后，此对象的属性不可改，而且也无法从此类派生出新子类。



不可变的“类”有何用？

“不可变类”可以方便和安全地用于多线程环境中，访问它们可以不用加锁，因而能提供较高的性能。

JDK中的实例：String

不可变类的实例：Address.java

String类相当复杂，Address类则展示了最基本的实现只读的编程技巧，学会容易得多。

编写自己的不可变类

要点

```
public final class Address {
    private final String detail;
    private final String postCode;

    //在构造方法里初始化两个实例属性
    public Address() {
        this(detail: "", postCode: "");
    }

    public Address(String detail, String postCode) {
        this.detail = detail;
        this.postCode = postCode;
    }

    //仅为两个实例属性提供get方法
    public String getDetail() {
        return this.detail;
    }

    public String getPostCode() {
        return this.postCode;
    }
}
```

- (1) 使用final定义类和字段
- (2) 字段仅提供get方法

```
//重写equals方法, 判断两个对象是否相等。
public boolean equals(Object obj) {
    if (obj instanceof Address) {
        Address ad = (Address) obj;
        return detail.equals(ad.detail)
            && postCode.equals(ad.postCode);
    }
    return false;
}

public int hashCode() {
    return detail.hashCode() + postCode.hashCode();
}
```

探索技术的奥秘.....



参看ExplorationJDKSource.java示例

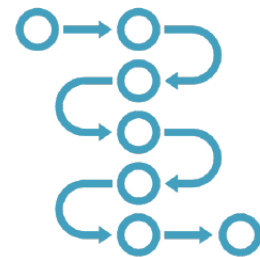
```
public class ExplorationJDKSource {  
    public static void main(String[] args) {  
        //输出: A@7ef20235  
        System.out.println(new A());  
    }  
}  
  
//类A没有定义任何的成员  
class A {  
}
```

我们得到了一个奇特的运行结果:

A@7ef20235



曲折的探索之路



- 请按照以下步骤进行技术探险：

- (1) 使用javap -c命令（或使用诸如jclasslib这样的工具），反汇编ExplorationJDKSource.class

- (2) 阅读字节码指令，弄明白println()那条语句到底调用了什么？

- (3) 依据第（2）得到的结论，使用IntelliJ打开JDK源码，查看真正被执行的代码是什么

- 激动啊，我终于发现了……

真相是什么！

源码之前，了无秘密

1 前面示例中，main方法实际上调用的是：

```
public void println(Object x)
```

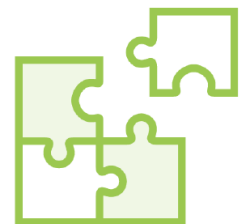
2 这一方法内部调用了String类的valueOf方法。valueOf方法内部又调用Object.toString方法，此方法的源码如下：

```
public String toString() {  
    return getClass().getName() +  
        "@ " + Integer.toHexString(hashCode());  
}
```

3 继续跟踪，发现hashCode方法是本地方法，由JVM开发者提供具体实现：

```
public native int hashCode();
```

阶段小结



- 现在请暂停一下学习的脚步，思索一下你到底学到了什么。
- 如果你没体会到什么，那么我可以肯定，你一定没有动手去做前面PPT中要你做
的事。
- 再次强调：



不亲自动手，只看不做，什么也学不会！
如果你真是这样，那后面的课程，也就没有必要继续
学下去了。

仍决定继续学?

OK, Let's Go!

子类与父类方法间的关系

由于Java并未对子类方法的命名做过多的限制，因此，子类与父类各自定义的方法之间，可以出现以下三种情况：

1. **扩充 (Extend)**：子类定义的方法，父类没有同名方法
2. **覆盖/重写 (Override)**：子类父类定义了完全一样的方法
3. **重载 (Overload)**：子类有父类的同名方法，但两者的参数类型或参数数目不一样

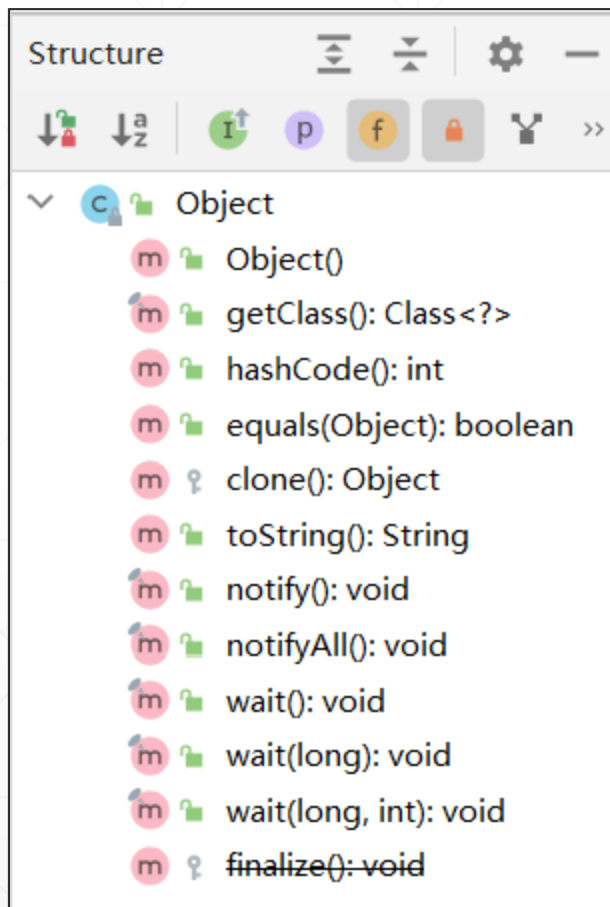
子类调用父类的构造方法

```
class Shape{  
    private final int top;  
    private final int left;  
    //基类的构造方法  
    public Shape(int top, int left) {  
        this.top = top;  
        this.left = left;  
    }  
}
```

本例中，子类与父类的构造方法，构成重载（Overload）关系。

```
class Rectangle extends Shape{  
    private final int width;  
    private final int height;  
  
    public Rectangle(int width,int height){  
        //调用本类的构造方法  
        this( top: 0, left: 0,width,height);  
    }  
  
    public Rectangle(int top, int left,  
                      int width, int height) {  
        super(top, left); //调用父类的构造方法  
        this.width = width;  
        this.height = height;  
    }  
}
```

顶层基类Object



在Java中，所有的类都直接或间接地派生自最顶层的基类Object，此类定义了一些方法，如左图所示。

在实际开发中，经常会“重写（Override）”Object类提供的特定方法，比如toString()、equals()等，在后面的课程中可以看到相关的实例。

重写Object方法示例：神奇的“+”号

我们来看一段代码（示例Fruit.java）：

```
public class Fruit {  
    public String toString() {  
        return "Fruit toString.";  
    }  
  
    public static void main(String[] args) {  
        Fruit f = new Fruit();  
        System.out.println("f = " + f);  
    }  
}
```

Fruit类重写了Object类中的
定义的toString()方法

注意最后一句，一个字串和一个对象“相加”，得到以下结果：

```
f = Fruit toString.
```

?

示例辨析



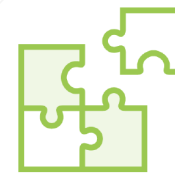
在“+”运算中，当一个对象与一个String对象连接时，会隐式地调用其toString()方法，默认情况下，此方法返回“类名 @ + hashCode”。

为了返回有意义的信息，子类可以重写toString()方法。

前页的示例中，Fruit类重写了Object类的toString方法，所以，示例输出了有意义的信息：

```
f = Fruit toString.
```

“方法重写/覆盖 (override)”的要点



- 方法重写要求子类与父类的方法一模一样，否则就是方法重载 (overload) ！
- 请自行编写代码测试以下特性：



在子类中，若要调用父类中被重写（或覆盖）的方法，可以使用`super`关键字。

Java“方法覆盖/重写”的语法规则



1. 覆盖/重写方法的允许访问范围不能小于原方法。
2. 覆盖/重写方法所抛出的异常不能比原方法更多。
3. 声明为final方法不允许覆盖/重写。例如，Object的getClass()方法不能覆盖/重写。
4. 不能覆盖/重写静态方法。

关于“继承”需要注意的-1



当子类重写基类的方法时，注意不要彻底改变方法的职责。子类应该是对基类同名方法行为特性的“调整”或“增强”。



尽量少写依据变量类型（是父类还是子类）进行判断的条件语句，而应该采用“多态”来简化代码。



避免继承的层次的过深，也就是说，从子类到最顶层的基类的“中间”类的个数不要太多，自定义类的继承层次最好限制在三层以下。

关于“继承”需要注意的-2

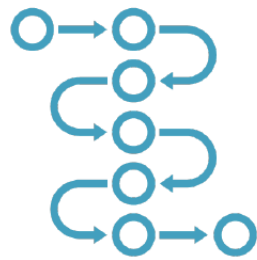


当类所对应的客观事物之间存在着“是一种 (IS_A)”关系时，为这些类之间建立继承关系才是合理的。



基类中可以保存一些所有子类都要用的公用代码，但不要仅为“代码重用”而在类型之间建立继承关系，考虑“组合”取代“继承”。

下一讲



在了解了继承相关的知识之后，我们将进一步学习Java面向对象特性的另一个重要内容——抽象基类和接口，并在此基础上，理解面向对象编程中至关重要的一个特性

多态